Faculty of Engineering & Technology
Department of Electrical & Computer Engineering
ENEE5304
INFORMATION AND CODING THEORY
Assignment 2: Source Encoding

Prepared by:
Saja Asfour 1210737

Instructor:
Dr. Wael Hashlamoun

Section: 1
Date: January 16, 2026

Table of Contents

List Of Figure....2
Input Sequence....3
The python Code:....3
The Result:....4
Part a: Find the number of bits needed to encode the sequence if ASCII code is used.....4
The python Code:....4
The Result:....4
Part b: Find the number of bits needed to encode the sequence if fixed length encoding is used. 4
The python Code:....4
The Result:....5
Part c: Find the source entropy in bits/character....5
The python Code:....5
The Result:....5
Part d: Find the Huffman code for the characters in the sequence 6
The python Code:....6
The Result:....6
Part e: Find the efficiency of the Huffman code relative to the entropy....6
The python Code:....6
The Result:....7
Part f: Find the number of bits needed to encode the sequence if Huffman code is used. Compare this to the ASCII code of Part a....7
The python Code:....7
The Result:....7
Part g: Find the number of bits needed to encode the sequence if Lempel-Ziv code is used. Compare this to the ASCII code of Part a and the Huffman code of Part f....8
The python Code:....8
The Result:....9
Comparison Summary: 10

List Of Figure

Figure 1:Code for the input sequence and helper function 3
Figure 2:Result of the input sequence information 4
Figure 3: python code for part a 4
Figure 4: Part a result 4
Figure 5:python code for part b 4
Figure 6:Part b result 5
Figure 7:python code for part c 5
Figure 8:Part c result 5
Figure 9:python code for part d 6
Figure 10:Part d result 6
Figure 11:python code for part e 6
Figure 12:Part e result 7
Figure 13:python code for part f 7
Figure 14:Part f result 7
Figure 15: python code for part g 8
Figure 16:Part g result 10

Input Sequence

Consider the random sequence of characters:

ACBAAACDBACABAEACACBABDACAEABACBACADBACABEAACABDACBAC
ACBAABACACDACBAEABACABDACACBAEABACBACDACABEAACBACABDA
CABACBACADBACABEAACABACDACBACABAEABACBACDABACABEAACBA
DACBACABACDACBAEABACBACABDACABEAACBACDACABAEABACBACD
ABACABEAACBACABDACBACAFABACBACDACABAEABACBACABDA

The python Code:

```python
import math
import heapq
from collections import Counter
import sys
sys.stdout.reconfigure(encoding="utf-8")


# --------------------------
# Input sequence (remove newlines)
# --------------------------
sequence = """
ACBAAACDBACABAEACACBABDACAEABACBACADBACABEAACABDACBAC
ACBAABACACDACBAEABACABDACACBAEABACBACDACABEAACBACABDA
CABACBACADBACABEAACABACDACBACABAEABACBACDABACABEAACBA
DACBACABACDACBAEABACBACABDACABEAACBACDACABAEABACBACD
ABACABEAACBACABDACBACAFABACBACDACABAEABACBACABDA
""".replace("\n", "").replace(" ", "")

# ============================================================
# Helpers
# ============================================================

Tabnine | Edit | Test | Explain | Document
def print_header(title: str):
    print("\n" + "=" * 70)
    print(title)
    print("=" * 70)

Tabnine | Edit | Test | Explain | Document
def safe_log2(x: int) -> float:
    return math.log2(x) if x > 0 else 0.0

Tabnine | Edit | Test | Explain | Document
def remove_empty_last_phrase(phrases):
    if phrases and phrases[-1][1] == "":
        return phrases[:-1]
    return phrases
```

```python
# ============================================================
# Main
# ============================================================

print_header("Sequence Info")
print("Sequence:", sequence)
print("Length (characters):", len(sequence))

counter = Counter(sequence)
symbols = sorted(counter.keys())
n = len(sequence)

print("\nSymbol counts:")
for s in symbols:
    print(f"  {s}: {counter[s]}")
print("Unique symbols:", symbols)
print("Number of unique symbols:", len(symbols))
```

Figure 1:Code for the input sequence and helper function

The Result:

```
==========================================================================
Sequence Info
==========================================================================
Sequence:
ACBAAACDBACABAEACACBABDACAEABACBACADBACABEAACABDACBACACBAABACACDACBAEABACABDACACBAEABACBACDACABEAACBACABDACABACBACADBACABEAACABACDACBACABAEABACBACDABACABEAACBADACBACABACDACBAEABACBACABDACABEAACB
ACDACABAEABACBACDABACABEAACBACABDACBACAFABACBACDACABAEABACBACABDA
Length (characters): 259

Symbol counts:
  A: 111
  B: 56
  C: 58
  D: 19
  E: 14
  F: 1
Unique symbols: ['A', 'B', 'C', 'D', 'E', 'F']
Number of unique symbols: 6
```

Figure 2:Result of the input sequence information

Part a: Find the number of bits needed to encode the sequence if ASCII code is used.

The python Code:

```python
# ---------------------------
# (a) ASCII bits
# ---------------------------
print_header("(a) Bits needed using ASCII")
ascii_bits_per_char = 8
ascii_total_bits = n * ascii_bits_per_char
print(f"ASCII uses {ascii_bits_per_char} bits/character.")
print(f"Total bits = length * 8 = {n} * 8 = {ascii_total_bits} bits")
```

Figure 3: python code for part a

The Result:

```
==========================================================================
(a) Bits needed using ASCII
==========================================================================
ASCII uses 8 bits/character.
Total bits = length * 8 = 259 * 8 = 2072 bits
```

Figure 4: Part a result

ASCII assigns a fixed 8-bit code to every character regardless of its frequency.
This method is simple but inefficient for data with a small alphabet.

Part b: Find the number of bits needed to encode the sequence if fixed length encoding is used.

The python Code:

```python
# ---------------------------
# (b) Fixed length bits
# ---------------------------
print_header("(b) Bits needed using Fixed-Length encoding")
k = len(symbols)
fixed_bits_per_char = math.ceil(safe_log2(k))
fixed_total_bits = n * fixed_bits_per_char

print(f"Number of symbols k = {k}")
print(f"Bits/character = ceil(log2(k)) = ceil(log2({k})) = {fixed_bits_per_char}")
print(f"Total bits = n * bits/char = {n} * {fixed_bits_per_char} = {fixed_total_bits} bits")
```

Figure 5:python code for part b

The Result:

```
======================================================================
(b) Bits needed using Fixed-Length encoding
======================================================================
Number of symbols k = 6
Bits/character = ceil(log2(k)) = ceil(log2(6)) = 3
Total bits = n * bits/char = 259 * 3 = 777 bits
```

Figure 6:Part b result

Fixed-length encoding assigns the same number of bits to all symbols.
It is more efficient than ASCII but still ignores symbol probabilities.

Part c: Find the source entropy in bits/character

The python Code:

```python
# ---------------------------
# (c) Entropy
# ---------------------------
print_header("(c) Source Entropy (bits/character)")
H = source_entropy(counter, n)
print("Probabilities:")
for s in symbols:
    p = counter[s] / n
    print(f"  P({s}) = {counter[s]}/{n} = {p:.6f}")

print(f"\nEntropy H = -Σ p(x) log2 p(x) = {H:.6f} bits/character")
```

Figure 7:python code for part c

The Result:

```
======================================================================
(c) Source Entropy (bits/character)
======================================================================
Probabilities:
  P(A) = 111/259 = 0.428571
  P(B) = 56/259 = 0.216216
  P(C) = 58/259 = 0.223938
  P(D) = 19/259 = 0.073359
  P(E) = 14/259 = 0.054054
  P(F) = 1/259 = 0.003861

Entropy H = -Σ p(x) log2 p(x) = 2.020019 bits/character
```

Figure 8:Part c result

Source entropy measures the average information content per symbol. It's calculated using Shannon's entropy formula:
$H(X) = -\Sigma P(x_i) \times \log_2(P(x_i))$
Where $P(x_i)$ is the probability of each character. The entropy represents the theoretical lower bound for lossless compression. A lower entropy indicates more predictability in the source.

Part d: Find the Huffman code for the characters in the sequence

The python Code:

```python
# ---------------------------
# (d) Huffman codes
# ---------------------------
print_header("(d) Huffman codes for the characters")
print("Building Huffman tree (merge steps):")
root = build_huffman(counter)

codes = extract_codes(root)
print("\nHuffman codes:")
for s in symbols:
    print(f"  {s}: {codes[s]}  (length={len(codes[s])})")
```

Figure 9:python code for part d

The Result:

```
======================================================================
(d) Huffman codes for the characters
======================================================================
Building Huffman tree (merge steps):
[Huffman merge step 1] merge (F,1) + (E,14) -> freq=15
[Huffman merge step 2] merge (None,15) + (D,19) -> freq=34
[Huffman merge step 3] merge (None,34) + (B,56) -> freq=90
[Huffman merge step 4] merge (C,58) + (None,90) -> freq=148
[Huffman merge step 5] merge (A,111) + (None,148) -> freq=259

Huffman codes:
  A: 0  (length=1)
  B: 111  (length=3)
  C: 10  (length=2)
  D: 1101  (length=4)
  E: 11001  (length=5)
  F: 11000  (length=5)
```

Figure 10:Part d result

Huffman coding is an optimal prefix-free code that assigns shorter codes to more frequent characters. The algorithm works by:

1. Creating leaf nodes for each character with its frequency
2. Repeatedly merging the two lowest-frequency nodes
3. Building a binary tree where left edges are '0' and right edges are '1'
4. The code for each character is the path from root to leaf

This ensures no code is a prefix of another, allowing unambiguous decoding.

Part e: Find the efficiency of the Huffman code relative to the entropy.

The python Code:

```python
# ---------------------------
# (e) Huffman efficiency relative to entropy
# ---------------------------
print_header("(e) Efficiency of Huffman relative to entropy")
L_avg = avg_code_length(counter, codes, n)
eff = H / L_avg if L_avg > 0 else 0.0

print(f"Average Huffman code length L = Σ p(x)*l(x) = {L_avg:.6f} bits/character")
print(f"Entropy H = {H:.6f} bits/character")
print(f"Efficiency η = H / L = {H:.6f} / {L_avg:.6f} = {eff:.6f} (={eff*100:.2f}%)")
```

Figure 11:python code for part e

The Result:

```
======================================================================
(e) Efficiency of Huffman relative to entropy
======================================================================
Average Huffman code length L = Σ p(x)*l(x) = 2.108108 bits/character
Entropy H = 2.020019 bits/character
Efficiency η = H / L = 2.020019 / 2.108108 = 0.958214 (=95.82%)
```

Figure 12:Part e result

Efficiency measures how close the Huffman code is to the theoretical entropy limit:
Efficiency = (H(X) / L_avg) × 100%
Where: H(X) is the source entropy , L_avg is the average code length
A higher efficiency (closer to 100%) indicates better compression.

Part f: Find the number of bits needed to encode the sequence if Huffman code is used. Compare this to the ASCII code of Part a.

The python Code:

```python
# --------------------------
# (f) Huffman total bits and compare to ASCII
# --------------------------
print_header("(f) Bits needed using Huffman code + comparison to ASCII")
huff_total_bits = total_bits_huffman(sequence, codes)

print("Per-symbol contribution:")
for s in symbols:
    bits_sym = counter[s] * len(codes[s])
    print(f"  {s}: count={counter[s]}, code_len={len(codes[s])}, bits={counter[s]}*{len(codes[s])}={bits_sym}")

print(f"\nTotal Huffman bits = Σ count(x)*l(x) = {huff_total_bits} bits")
print(f"ASCII total bits (from part a) = {ascii_total_bits} bits")

saved_vs_ascii = ascii_total_bits - huff_total_bits
print(f"Bits saved vs ASCII = {ascii_total_bits} - {huff_total_bits} = {saved_vs_ascii} bits")
print(f"Compression ratio vs ASCII = Huffman/ASCII = {huff_total_bits}/{ascii_total_bits} = {huff_total_bits/ascii_total_bits:.6f}")
```

Figure 13:python code for part f

The Result:

```
======================================================================
(f) Bits needed using Huffman code + comparison to ASCII
======================================================================
Per-symbol contribution:
  A: count=111, code_len=1, bits=111*1=111
  B: count=56, code_len=3, bits=56*3=168
  C: count=58, code_len=2, bits=58*2=116
  D: count=19, code_len=4, bits=19*4=76
  E: count=14, code_len=5, bits=14*5=70
  F: count=1, code_len=5, bits=1*5=5

Total Huffman bits = Σ count(x)*l(x) = 546 bits
ASCII total bits (from part a) = 2072 bits
Bits saved vs ASCII = 2072 - 546 = 1526 bits
Compression ratio vs ASCII = Huffman/ASCII = 546/2072 = 0.263514
```

Figure 14:Part f result

To encode the entire sequence with Huffman coding, we multiply each character's code length by its frequency:
Total bits = Σ (frequency × code_length)
Comparing to ASCII:
- ASCII: 2072 bits
- Huffman: 564 bits
- Savings: 1526

Huffman coding exploits the non-uniform distribution of characters, assigning shorter codes to frequent characters and longer codes to rare ones .

Part g: Find the number of bits needed to encode the sequence if Lempel-Ziv code is used. Compare this to the ASCII code of Part a and the Huffman code of Part f.

The python Code:

```python
# ---------------------------
# (g) Lempel-Ziv (LZ78) bits and comparison
# ---------------------------
print_header("(g) Bits needed using Lempel-Ziv (LZ78) + comparisons")

print("LZ78 encoding steps:")
phrases, lz_dict = lz78_encode(sequence)
phrases = remove_empty_last_phrase(phrases)

print("\nLZ78 phrases (index, char):")
for i, (idx, ch) in enumerate(phrases, start=1):
    print(f"  {i:>3}: ({idx}, '{ch}')")

final_dict_size = len(phrases) + 1   # +1 for empty string
print(f"\nNumber of phrases = {len(phrases)}")
print(f"Final dictionary size (including empty) = {final_dict_size}")

# ============================================================
# (g-1) LZ78 with FIXED-LENGTH symbols (ceil(log2(k)))
# ============================================================
alphabet_bits_fixed = math.ceil(safe_log2(k))
print_header("(g-1) LZ78 using fixed-length symbols")
print(f"Character bits = ceil(log2(k)) = ceil(log2({k})) = {alphabet_bits_fixed} bits")

lz_total_fixed_fixed, fixed_idx_bits_fixed, fixed_breakdown_fixed = \
    lz78_bits_fixed_final(phrases, alphabet_bits_fixed, final_dict_size)

print(f"Index bits = ceil(log2(final_dict_size)) = {fixed_idx_bits_fixed} bits")
print("\nFixed-bit accounting:")

print(f"\nTotal LZ78 bits (fixed index, fixed-length symbols) = {lz_total_fixed_fixed} bits")

# ============================================================
# (g-2) LZ78 with ASCII symbols (8 bits per character)
# ============================================================
alphabet_bits_ascii = 8
print_header("(g-2) LZ78 using ASCII symbols")
print(f"Character bits = {alphabet_bits_ascii} bits (ASCII)")

lz_total_fixed_ascii, fixed_idx_bits_ascii, fixed_breakdown_ascii = \
    lz78_bits_fixed_final(phrases, alphabet_bits_ascii, final_dict_size)

print(f"Index bits = ceil(log2(final_dict_size)) = {fixed_idx_bits_ascii} bits")
print("\nFixed-bit accounting:")

print(f"\nTotal LZ78 bits (fixed index, ASCII symbols) = {lz_total_fixed_ascii} bits")

# ============================================================
# Final comparison
# ============================================================
print_header("(g) Final comparison")
print(f"ASCII total bits   = {ascii_total_bits}")
print(f"Huffman total bits = {huff_total_bits}")

print("\nLZ78 totals:")
print(f"  Fixed-length symbols : {lz_total_fixed_fixed} bits")
print(f"  ASCII symbols        : {lz_total_fixed_ascii} bits")

print("\nCompression ratios vs ASCII:")
print(f"  LZ78 (fixed-length symbols) / ASCII = {lz_total_fixed_fixed/ascii_total_bits:.6f}")
print(f"  LZ78 (ASCII symbols)        / ASCII = {lz_total_fixed_ascii/ascii_total_bits:.6f}")

print("\nCompression ratios vs Huffman:")
print(f"  LZ78 (fixed-length symbols) / Huffman = {lz_total_fixed_fixed/huff_total_bits:.6f}")
print(f"  LZ78 (ASCII symbols)        / Huffman = {lz_total_fixed_ascii/huff_total_bits:.6f}")
```

Figure 15: python code for part g

The Result:

```
OUTPUT
==================================================================
(g) Bits needed using Lempel-Ziv (LZ78) + comparisons
==================================================================
LZ78 encoding steps:
[LZ78 step 1] New phrase 'A' -> index 1. Output (0, 'A')
[LZ78 step 2] New phrase 'C' -> index 2. Output (0, 'C')
[LZ78 step 3] New phrase 'B' -> index 3. Output (0, 'B')
[LZ78 step 4] New phrase 'AA' -> index 4. Output (1, 'A')
[LZ78 step 5] New phrase 'AC' -> index 5. Output (1, 'C')
[LZ78 step 6] New phrase 'D' -> index 6. Output (0, 'D')
[LZ78 step 7] New phrase 'BA' -> index 7. Output (3, 'A')
[LZ78 step 8] New phrase 'CA' -> index 8. Output (2, 'A')
[LZ78 step 9] New phrase 'BAE' -> index 9. Output (7, 'E')
[LZ78 step 10] New phrase 'ACA' -> index 10. Output (5, 'A')
[LZ78 step 11] New phrase 'CB' -> index 11. Output (2, 'B')
[LZ78 step 12] New phrase 'AB' -> index 12. Output (1, 'B')
[LZ78 step 13] New phrase 'DA' -> index 13. Output (6, 'A')
[LZ78 step 14] New phrase 'CAE' -> index 14. Output (8, 'E')
[LZ78 step 15] New phrase 'ABA' -> index 15. Output (12, 'A')
[LZ78 step 16] New phrase 'CBA' -> index 16. Output (11, 'A')
[LZ78 step 17] New phrase 'CAD' -> index 17. Output (8, 'D')
[LZ78 step 18] New phrase 'BAC' -> index 18. Output (7, 'C')
[LZ78 step 19] New phrase 'ABE' -> index 19. Output (12, 'E')
[LZ78 step 20] New phrase 'AAC' -> index 20. Output (4, 'C')
[LZ78 step 21] New phrase 'ABD' -> index 21. Output (12, 'D')
[LZ78 step 22] New phrase 'ACB' -> index 22. Output (5, 'B')
[LZ78 step 23] New phrase 'ACAC' -> index 23. Output (10, 'C')
[LZ78 step 24] New phrase 'BAA' -> index 24. Output (7, 'A')
[LZ78 step 25] New phrase 'BACA' -> index 25. Output (18, 'A')
[LZ78 step 26] New phrase 'CD' -> index 26. Output (2, 'D')
[LZ78 step 27] New phrase 'ACBA' -> index 27. Output (22, 'A')
[LZ78 step 28] New phrase 'E' -> index 28. Output (0, 'E')
[LZ78 step 29] New phrase 'ABAC' -> index 29. Output (15, 'C')
[LZ78 step 30] New phrase 'ABDA' -> index 30. Output (21, 'A')
[LZ78 step 31] New phrase 'CAC' -> index 31. Output (8, 'C')
[LZ78 step 32] New phrase 'BAEA' -> index 32. Output (9, 'A')
[LZ78 step 33] New phrase 'BACB' -> index 33. Output (18, 'B')
[LZ78 step 34] New phrase 'ACD' -> index 34. Output (5, 'D')
[LZ78 step 35] New phrase 'ACAB' -> index 35. Output (10, 'B')
[LZ78 step 36] New phrase 'EA' -> index 36. Output (28, 'A')
[LZ78 step 37] New phrase 'ACBAC' -> index 37. Output (27, 'C')
[LZ78 step 38] New phrase 'ABDAC' -> index 38. Output (30, 'C')
[LZ78 step 39] New phrase 'ABACB' -> index 39. Output (29, 'B')
[LZ78 step 40] New phrase 'ACAD' -> index 40. Output (10, 'D')
[LZ78 step 41] New phrase 'BACAB' -> index 41. Output (25, 'B')
[LZ78 step 42] New phrase 'EAA' -> index 42. Output (36, 'A')
[LZ78 step 43] New phrase 'CAB' -> index 43. Output (8, 'B')
[LZ78 step 44] New phrase 'ACDA' -> index 44. Output (34, 'A')
[LZ78 step 45] New phrase 'CBAC' -> index 45. Output (16, 'C')
[LZ78 step 46] New phrase 'ABAE' -> index 46. Output (15, 'E')
[LZ78 step 47] New phrase 'ABACBA' -> index 47. Output (39, 'A')
[LZ78 step 48] New phrase 'CDA' -> index 48. Output (26, 'A')
[LZ78 step 49] New phrase 'BACABE' -> index 49. Output (41, 'E')
[LZ78 step 50] New phrase 'AACB' -> index 50. Output (20, 'B')
```

```
[LZ78 step 50] New phrase 'AACB' -> index 50. Output (20, 'B')
[LZ78 step 51] New phrase 'AD' -> index 51. Output (1, 'D')
[LZ78 step 52] New phrase 'ACBACA' -> index 52. Output (37, 'A')
[LZ78 step 53] New phrase 'BACD' -> index 53. Output (18, 'D')
[LZ78 step 54] New phrase 'ACBAE' -> index 54. Output (27, 'E')
[LZ78 step 55] New phrase 'ABACBAC' -> index 55. Output (47, 'C')
[LZ78 step 56] New phrase 'ABDACA' -> index 56. Output (38, 'A')
[LZ78 step 57] New phrase 'BE' -> index 57. Output (3, 'E')
[LZ78 step 58] New phrase 'AACBA' -> index 58. Output (50, 'A')
[LZ78 step 59] New phrase 'CDAC' -> index 59. Output (48, 'C')
[LZ78 step 60] New phrase 'ABAEA' -> index 60. Output (46, 'A')
[LZ78 step 61] New phrase 'BACBA' -> index 61. Output (33, 'A')
[LZ78 step 62] New phrase 'CDAB' -> index 62. Output (48, 'B')
[LZ78 step 63] New phrase 'ACABE' -> index 63. Output (35, 'E')
[LZ78 step 64] New phrase 'AACBAC' -> index 64. Output (58, 'C')
[LZ78 step 65] New phrase 'ABDACB' -> index 65. Output (38, 'B')
[LZ78 step 66] New phrase 'ACAF' -> index 66. Output (10, 'F')
[LZ78 step 67] New phrase 'ABACBACD' -> index 67. Output (55, 'D')
[LZ78 step 68] New phrase 'ACABA' -> index 68. Output (35, 'A')
[LZ78 step 69] New phrase 'EAB' -> index 69. Output (36, 'B')
[LZ78 step 70] New phrase 'ACBACAB' -> index 70. Output (52, 'B')
[LZ78 final] leftover phrase 'DA' -> Output (13, '')

LZ78 phrases (index, char):
    1: (0, 'A')
    2: (0, 'C')
    3: (0, 'B')
    4: (1, 'A')
    5: (1, 'C')
    6: (0, 'D')
    7: (3, 'A')
    8: (2, 'A')
    9: (7, 'E')
   10: (5, 'A')
   11: (2, 'B')
   12: (1, 'B')
   13: (6, 'A')
   14: (8, 'E')
   15: (12, 'A')
   16: (11, 'A')
   17: (8, 'D')
   18: (7, 'C')
   19: (12, 'E')
   20: (4, 'C')
   21: (12, 'D')
   22: (5, 'B')
   23: (10, 'C')
   24: (7, 'A')
   25: (18, 'A')
   26: (2, 'D')
   27: (22, 'A')
   28: (0, 'E')
   29: (15, 'C')
   30: (21, 'A')
```

```
  31: (8, 'C')
  32: (9, 'A')
  33: (18, 'B')
  34: (5, 'D')
  35: (10, 'B')
  36: (28, 'A')
  37: (27, 'C')
  38: (30, 'C')
  39: (29, 'B')
  40: (10, 'D')
  41: (25, 'B')
  42: (36, 'A')
  43: (8, 'B')
  44: (34, 'A')
  45: (16, 'C')
  46: (15, 'E')
  47: (39, 'A')
  48: (26, 'A')
  49: (41, 'E')
  50: (20, 'B')
  51: (1, 'D')
  52: (37, 'A')
  53: (18, 'D')
  54: (27, 'E')
  55: (47, 'C')
  56: (38, 'A')
  57: (3, 'E')
  58: (50, 'A')
  59: (48, 'C')
  60: (46, 'A')
  61: (33, 'A')
  62: (48, 'B')
  63: (35, 'E')
  64: (58, 'C')
  65: (38, 'B')
  66: (10, 'F')
  67: (55, 'D')
  68: (35, 'A')
  69: (36, 'B')
  70: (52, 'B')

Number of phrases = 70
Final dictionary size (including empty) = 71

==================================================================
(g-1) LZ78 using fixed-length symbols
==================================================================
Character bits = ceil(log2(k)) = ceil(log2(6)) = 3 bits
Index bits = ceil(log2(final_dict_size)) = 7 bits

Total LZ78 bits (fixed index, fixed-length symbols) = 700 bits

==================================================================
(g-2) LZ78 using ASCII symbols
==================================================================
Character bits = 8 bits (ASCII)
Index bits = ceil(log2(final_dict_size)) = 7 bits

Total LZ78 bits (fixed index, ASCII symbols) = 1050 bits

==================================================================
(g) Final comparison
==================================================================
ASCII total bits   = 2072
Huffman total bits = 546

LZ78 totals:
  Fixed-length symbols : 700 bits
  ASCII symbols        : 1050 bits

Compression ratios vs ASCII:
  LZ78 (fixed-length symbols) / ASCII = 0.337838
  LZ78 (ASCII symbols)        / ASCII = 0.506757

Compression ratios vs Huffman:
  LZ78 (fixed-length symbols) / Huffman = 1.282051
  LZ78 (ASCII symbols)        / Huffman = 1.923077

[Done] exited with code=0 in 0.144 seconds
```

Figure 16:Part g result

In this part, the LZ78 algorithm is applied to the input sequence, producing 70 phrases and a final dictionary size of 71 entries. Two encoding schemes are considered. In the first scheme, characters are represented using fixed-length codes based on the source alphabet size, requiring 3 bits per character and 7 bits per dictionary index, which results in a total of 700 bits. In the second scheme, characters are encoded using ASCII with 8 bits per character while keeping the same 7-bit index, leading to a total of 1050 bits. The results show that using fixed-length symbol representation significantly improves LZ78 compression efficiency compared to ASCII, while both approaches outperform plain ASCII encoding but remain less efficient than Huffman coding for this sequence.

Comparison Summary:

For this sequence, Huffman coding is clearly the best-performing method. It compresses the data from 2072 bits to 546 bits, achieving a reduction of about **73.6%**, which highlights the strength of frequency-based compression when symbol usage is highly uneven. Lempel–Ziv (LZ78) also provides meaningful compression, but its performance depends on the symbol representation. Using fixed-length symbols, LZ78 reduces the data size by about **66.2%**, while using ASCII symbols yields a smaller reduction of about **49.3%**. Although LZ78 successfully exploits repeated patterns, Huffman coding remains more efficient for this

dataset, achieving lower overall bit usage. In summary, statistical methods like Huffman excel when symbol probabilities are well defined, whereas dictionary-based methods such as LZ78 are more flexible and better suited for scenarios involving unknown data statistics or streaming input.